

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Zeitplan bis zum Jahresende

08.01.2026	Datenflussanalyse	Übungsblatt „Datenflussanalyse“
13.01.2026	Datenflussanalyse von TRIPLA	
15.01.2026	Projekt IV (Kontrollflussanalyse)	Abgabe Übungsblatt „Datenflussanalyse“
20.01.2026	Code Clone Detection	
22.01.2026	Reverse Engineering, Erkennung von Entwurfsmustern	Abgabe Projekt IV
27.01.2026	Gastvortrag: „Programm Analysis and Transformation from a Practitioner’s Point of View“ (Jonas Eckstein, itestra)	Projekt V (Datenflussanalyse)
29.01.2026	Programmtransformationen	Übungsblatt „Programmtransformationen“
03.02.2026	Partielle Evaluation von TRIPLA	
05.02.2026	Higher-Order TRIPLA	
10.02.2026	Details zum Portfolio + Tipps zu Projekt V	Abgabe + Übung ÜTRAFO
17.03.2026	Abgabe Portfolio inkl. Projekt V	



Partielle Auswertung von TRIPLA

Bildquelle: mit KI erzeugt (Bing, DALL-E)

Partial Evaluation: Unfolding

```
pow(n, x) = if (n==0) then 1 else x*pow(n-1, x)
```

```
pow(3, x)  -> if (3==0) then 1 else x*pow(3-1, x)
           -> x*pow(2,x)
           -> x*(if (2==0) then 1 else x*pow(2-1, x))
           -> x*x*pow(1,x)
           -> x*x*(if (1==0) then 1 else x*pow(1-1, x))
           -> x*x*x*pow(0,x)
           -> x*x*x*(if (0==0) then 1 else x*pow(0-1, x))
           -> x*x*x*1
           -> x*x*x
```

```
pow(n, 3)  -> ???
```

Wie kann man
vorab berechnen,
ob partielle
Auswertung
terminiert ?

Partial Evaluation: Unfolding

```
while(x>0) { x=x-1;  
            y=y-1;  
            }  
return y+42;
```

Annahme: **x ist 3**

```
y=y-1  
y=y-1  
y=y-1  
return y+42;
```

```
while(x>0) { x=x+1;  
            y=y-1;  
            }  
return y+42;
```

Annahme: **x ist 3**

```
y=y-1  
y=y-1  
...
```

Ein Wert ist statisch (**static**), wenn er zum Zeitpunkt der partiellen Auswertung bekannt ist, ansonsten ist er dynamisch (**dynamic**).

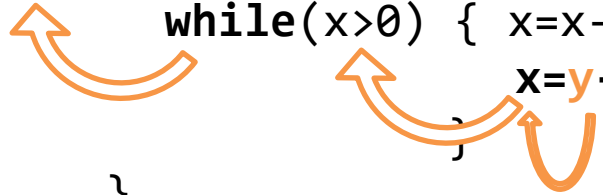
```
while(x>0) { x=x-1;
            while(x>0) { x=x-1;
                        y=y-1;
                        }
            }
return y+42;
```



Annahme: **x ist statisch**

Partielle Auswertung liefert
endliche Folge von Zuweisungen
gefolgt von einer return-Anweisung

```
while(x>0) { x=x-1;
            while(x>0) { x=x-1;
                        x=y-1;
                        }
            }
return y+42;
```



Annahme: **x ist statisch**

Partielle Auswertung liefert
ursprüngliches Programm

KONSTRUKT	REGEL / LOGIK	RESULTIERENDER STATUS
KONSTANTE	Wert ist zur Spezialisierungszeit fest	S (Static)
VARIABLE (INPUT)	Bekannt (S) oder Unbekannt (D)	S oder D
OPERATOR S OP S	Beide Operanden vorab berechenbar	S (Static)
OPERATOR S OP D	Ein Operand erst zur Laufzeit bekannt	D (Dynamic)
WHILE (S) DO	Bedingung statisch: Schleife wird entrollt	Reducible
WHILE (D) DO	Bedingung dynamisch: Schleife bleibt	Residual
ZUWEISUNG X := S	Variable erhält statischen Wert	x -> S
ZUWEISUNG X := D	Variable wird "dynamisch infiziert"	x -> D

Partielle Auswertung von TRIPLA

Zwei Phasen:

- Binding-Time Analysis **BTA** berechnet $\beta : P \rightarrow \{\text{static}, \text{dynamic}\}$
- Reduktion

$$peval(e^p, \rho) = reduce(e^p, \rho, \beta) \text{ where } (\rho', \beta) = bta(e^p, \alpha(\rho), \emptyset, \text{static})$$

- Abstrakter Wertebereich: $\text{static} \sqsubseteq \text{dynamic}$ und $\text{static} \sqcup \text{dynamic} = \text{dynamic}$.
- Abstrakte Bindungsumgebung

$$\alpha(\rho) = \rho' \quad \text{where } \rho'(x) = \begin{cases} \text{static} & \text{if } \rho(x) \text{ is defined} \\ \text{dynamic} & \text{if } \rho(x) \text{ is undefined} \end{cases}$$

We extend the definition of least upper bounds for environments $\rho : V \rightarrow \mathbb{Z}$

$$\rho_1 \sqcup \rho_2 = \rho_3 \quad \text{where } \rho_3(x) = \begin{cases} v & \text{if } \rho_1(x) = v = \rho_2(x) \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\rho_1 \sqsubseteq \rho_2 \quad \forall x : \rho_1(x) = v \Rightarrow \rho_2(x) = v$$

... and for abstract environments $\tilde{\rho} : V \rightarrow \{\text{static}, \text{dynamic}\}$

$$\tilde{\rho}_1 \sqcup \tilde{\rho}_2 = \tilde{\rho}_3 \quad \text{where } \tilde{\rho}_3(x) = \tilde{\rho}_1(x) \sqcup \tilde{\rho}_2(x)$$

$$\tilde{\rho}_1 \sqsubseteq \tilde{\rho}_2 \quad \forall x : \rho_1(x) \sqsubseteq \rho_2(x)$$

Binding-Time Analysis für Konstanten, Variablen, binäre Ausdrücke und Zuweisungen

$$\begin{aligned}
 bta(c^p, \tilde{\rho}, \beta, \kappa) &= (\tilde{\rho}, \beta[p/\text{static}]) \\
 bta(x^p, \tilde{\rho}, \beta, \kappa) &= \begin{cases} (\tilde{\rho}, \beta[p/\tilde{\rho}(x)]) & \text{if } \tilde{\rho}(x) \text{ is defined} \\ (\tilde{\rho}, \beta[p/\text{dynamic}]) & \text{otherwise} \end{cases} \\
 bta((e_1^{p_1} \text{ op } e_2^{p_2})^p, \tilde{\rho}, \beta, \kappa) &= (\tilde{\rho}_2, \beta_2[p/\beta_2(p_1) \sqcup \beta_2(p_2)]) \\
 &\quad \text{where } (\tilde{\rho}_1, \beta_1) = bta(e_1^{p_1}, \tilde{\rho}, \beta, \kappa) \\
 &\quad \text{and } (\tilde{\rho}_2, \beta_2) = bta(e_2^{p_2}, \tilde{\rho}_1, \beta_1, \kappa) \\
 bta((x=e^{p_1})^p, \tilde{\rho}, \beta, \kappa) &= (\tilde{\rho}_1[x/\beta_1(p_1) \sqcup \kappa], \beta_1[p/\beta_1(p_1)]) \\
 &\quad \text{where } (\tilde{\rho}_1, \beta_1) = bta(e^{p_1}, \tilde{\rho}, \beta, \kappa)
 \end{aligned}$$

Partial Evaluation für Konstanten, Variablen, binäre Ausdrücke und Zuweisungen

$$\begin{aligned}
 \text{reduce}(c^p, \rho, \beta) &= (c^p, \rho, \beta) && \text{Es kommen neue} \\
 \text{reduce}(x^p, \rho, \beta) &= \begin{cases} (\rho(x)^{p'}, \rho, \beta) & \text{if } \beta(p) = \text{static} \\ (x^p, \rho, \beta) & \text{otherwise} \end{cases} && \text{Programmpunkte hinzu!} \\
 &\text{where } p' \text{ is a new program-point label} \\
 \text{reduce}((e_1^{p_1} \text{ op } e_2^{p_2})^p, \rho, \beta) &= \begin{cases} (\overline{\text{op}}(v_1, v_2)^{p'}, \rho, \beta_2[p'/\text{static}]) & \text{if } \beta(p) = \text{static} \\ ((v_1 \text{ op } v_2)^{p'}, \rho, \beta_2[p'/\text{dynamic}]) & \text{otherwise} \end{cases} \\
 &\text{where } (v_1, \rho_1, \beta_1) = \text{reduce}(e_1^{p_1}, \rho, \beta) \\
 &\text{and } (v_2, \rho_2, \beta_2) = \text{reduce}(e_2^{p_2}, \rho_1, \beta_1) \\
 &\text{and } p' \text{ is a new program-point label} \\
 \text{reduce}((x=e^{p_1})^p, \rho, \beta) &= ((x=v^{p'})^{p^*}, \rho_2, \beta_1[p^*/\beta_1(p')]) \\
 &\text{where } (v^{p'}, \rho_1, \beta_1) = \text{reduce}(e^{p_1}, \rho, \beta) \\
 &\text{and } p^* \text{ is a new program-point label} \\
 &\text{and } \rho_2 = \begin{cases} (\rho_1[x/v]) & \text{if } \beta(p) = \text{static} \\ \rho_1 & \text{otherwise} \end{cases}
 \end{aligned}$$

Binding-Time Analysis für Kontrollflussanweisungen

$$bta((e_1^{p_1}; e_2^{p_2})^p, \tilde{\rho}, \beta, \kappa) = (\tilde{\rho}_2, \beta_1[p/\beta_2(p_1) \sqcup \beta_2(p_2)])$$

where $(\tilde{\rho}_1, \beta_1) = bta(e_1^{p_1}, \tilde{\rho}, \beta, \kappa)$
and $(\tilde{\rho}_2, \beta_2) = bta(e_2^{p_2}, \tilde{\rho}_1, \beta_1, \kappa)$

$$bta((\text{if } e_1^{p_1} \text{ then } e_2^{p_2} \text{ else } e_3^{p_3})^p, \tilde{\rho}, \beta, \kappa) = (\tilde{\rho}_2 \sqcup \tilde{\rho}_3, (\beta_2 \sqcup \beta_3)[p/\beta_1(p_1)])$$

where $(\tilde{\rho}_1, \beta_1) = bta(e_1^{p_1}, \tilde{\rho}, \beta, \kappa)$
and $\kappa' = \kappa \sqcup \beta_1(p_1)$
and $(\tilde{\rho}_2, \beta_2) = bta(e_2^{p_2}, \tilde{\rho}_1, \beta_1, \kappa')$
and $(\tilde{\rho}_3, \beta_3) = bta(e_3^{p_3}, \tilde{\rho}_1, \beta_1, \kappa')$

$$bta((\text{while } e_1^{p_1} \text{ do } e_2^{p_2})^p, \rho, \kappa) = (\tilde{\rho}^k, \beta^k[p/\beta^k(p_1)]) \text{ for the smallest } k$$

such that $\tilde{\rho}^k = \tilde{\rho}^{k+1}$ and $\beta^i = \beta^{k+1}$
where $\tilde{\rho}^0 = \tilde{\rho}$, $\beta^0 = \beta$, $\kappa^0 = \kappa$ and for all $i > 0$:

$(\tilde{\rho}_1^i, \beta_1^i) = bta(e_1^{p_1}, \tilde{\rho}^{i-1}, \beta^{i-1}, \kappa^{i-1})$
 $\kappa^i = \kappa^{i-1} \sqcup \beta_1(p_1)$
 $(\tilde{\rho}_2^i, \beta_2^i) = bta(e_2^{p_2}, \tilde{\rho}_1^i, \beta_1^i, \kappa^i)$
 $\tilde{\rho}^i = \tilde{\rho}_2^i \sqcup \tilde{\rho}^{i-1}$ and $\beta^i = \beta_2^i \sqcup \beta^{i-1}$

Partial Evaluation für Kontrollflussanweisungen

$$\text{reduce}((e_1^{p_1}; e_2^{p_2})^p, \rho, \beta) = \begin{cases} (v_1^{p'}, \rho_2, \beta_2) & \text{if } \beta(p) = \text{static} \text{ and } v_2 = \emptyset \\ (v_2^{p''}, \rho_2, \beta_2) & \text{if } \beta(p) = \text{static} \text{ and } v_2 \neq \emptyset \\ ((v_1^{p'}; v_2^{p''})^{p^*}, \rho_2, \beta_3) & \text{otherwise} \end{cases}$$

where $(v_1^{p'}, \rho_1, \beta_1) = \text{reduce}(e_1^{p_1}, \rho, \beta)$
 and $(v_2^{p''}, \rho_2, \beta_2) = \text{reduce}(e_2^{p_2}, \rho_1, \beta_1)$
 and $\beta_3 = \beta_2[p^* / \beta_2(p') \sqcup \beta_2(p'')]$
 and p^* is a new program-point label

Partial Evaluation für Kontrollflussanweisungen

$$\begin{aligned}
 & \text{reduce}((\text{if } e_1^{p_1} \text{ then } e_2^{p_2} \text{ else } e_3^{p_3})^p, \rho, \beta) = \\
 & \left\{ \begin{array}{ll} (v_2^{p''}, \rho_2, \beta_2) & \text{if } \beta(p) = \text{static and } v_1 = \text{true} \\ (v_3^{p^\circ}, \rho_3, \beta_3) & \text{if } \beta(p) = \text{static and } v_1 = \text{false} \\ ((\text{if } v_1^{p'} \text{ then } v_2^{p''} \text{ else } v_3^{p^\circ})^{p^*}, & \text{otherwise} \\ \rho_2 \sqcup \rho_3, (\beta_2 \sqcup \beta_3)[p^* / \beta_1(p')]) & \end{array} \right.
 \end{aligned}$$

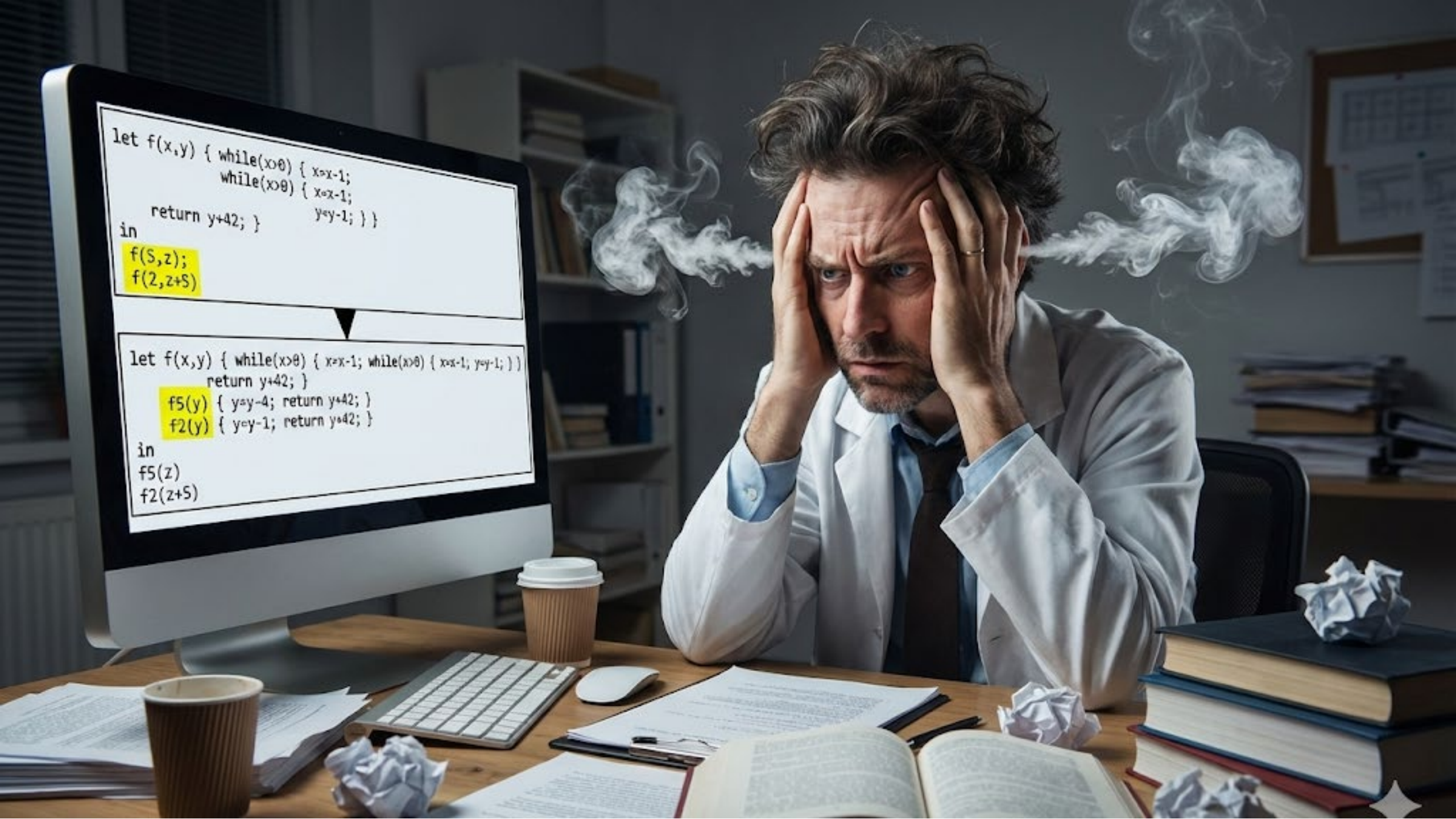
where $(v_1^{p'}, \rho_1, \beta_1) = \text{reduce}(e_1^{p_1}, \rho, \beta)$

and $(v_2^{p''}, \rho_2, \beta_2) = \text{reduce}(e_2^{p_2}, \rho_1, \beta_1)$

and $(v_3^{p^\circ}, \rho_3, \beta_3) = \text{reduce}(e_3^{p_3}, \rho_1, \beta_1)$

Partial Evaluation für Kontrollflussanweisungen

$$\begin{aligned}
 & \text{reduce}((\text{while } e_1^{p_1} \text{ do } e_2^{p_2})^p, \rho, \beta) = \\
 & \begin{cases} \text{reduce}((e_2^{p_2}; \text{while } e_1^{p_1} \text{ do } e_2^{p_2})^{p^*}, \rho_1, \beta_1[p^* / \beta_1(p_2)]) & \text{if } \beta(p) = \text{static} \text{ and } v_1 = \text{true} \\ (\emptyset, \rho_1, \beta_1) & \text{if } \beta(p) = \text{static} \text{ and } v_1 = \text{false} \\ ((\text{while } v_1^{p'} \text{ do } v_2^{p''})^{p^*}, \rho_1 \sqcup \rho_2, \beta_2[p^* / \text{dynamic}]) & \text{otherwise} \end{cases} \\
 & \text{where } (v_1^{p'}, \rho_1, \beta_1) = \text{reduce}(e_1^{p_1}, \rho, \beta) \\
 & \text{and } (v_2^{p''}, \rho_2, \beta_2) = \text{reduce}(e_2^{p_2}, \rho_1, \beta_1)
 \end{aligned}$$



```
let f(x,y) { while(x>0) { x=x-1;  
                  while(x>0) { x=x-1;  
                        return y+42; }      y=y-1; } }
```

```
in  
f(S,z);  
f(2,z+S)
```

```
let f(x,y) { while(x>0) { x=x-1; while(x>0) { x=x-1; y=y-1; } }  
            return y+42; }
```

```
    f5(y) { y=y-4; return y+42; }  
    f2(y) { y=y-1; return y+42; }
```

```
in  
f5(z)  
f2(z+S)
```